



ADDING PAYMENT TO YOUR EXPERIENCE

DRAFT

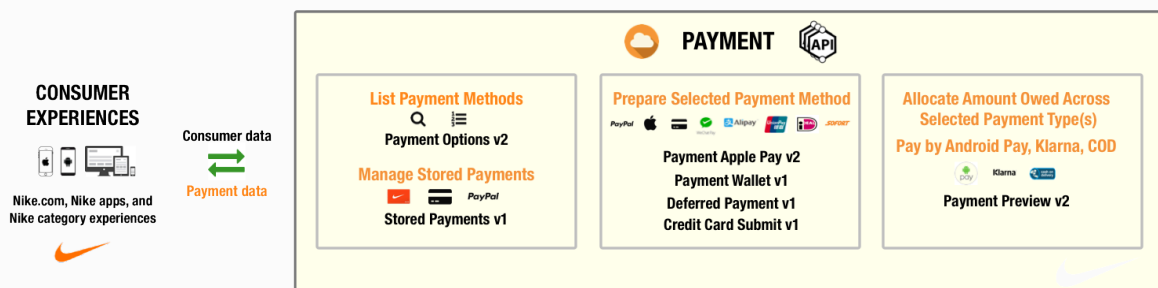
[DOWNLOAD](#)

Last Updated: 2/21/2019

Manage the payment process for customers purchasing Nike products and services.

TIPS:

- Before using this guide, read [Using NDe APIs](#) and [Payment Overview](#).
- Use this Developer's Guide as a supplement to the API Reference for detailed use cases. See [API Quick Reference](#) for links to all the API Reference docs discussed in this guide.
- The steps involving **Checkout** are covered in [Adding Cart & Checkout to Your Experience](#).



The payment process during Checkout consists of four steps:

1 Listing Payment Methods and Managing Stored Payments

Your experience can get a list of stored payments for a logged in consumer by calling the [Stored Payments](#) service. The Stored Payments service can also be used to add, delete, and update a consumer's stored payments, including the default stored payment. The [Payment Options](#) service lists and validates non-stored payments.

2 Preparing payment for purchase

Depending upon the payment type, your experience will need to perform different actions to prepare the payment for purchase. Before a customer can pay with [Apple Pay](#), an Apple Pay session must be started. To allow customers to pay in the PayPal Express or PayPal Mark flows, you will need to call the [Wallet Payment](#) service to start a PayPal session. When paying by a non-stored credit card, your experience will need to collect the customer's credit card information using the [Credit Card Submit](#) service. If customers pay by a [Deferred Payment](#) type such as Alipay or WeChat, your experience will need to generate a signed URL and redirect the customer so they can pay at the vendor's site after they submit the Nike Checkout.

3 Payment Preview

Because customers can pay for their Checkout using Gift Cards, Vouchers, and another payment type, it is necessary to calculate how much of the Checkout will be paid by each payment type by calling [Payment Preview](#). Your experience can display the payment allocation results to customers so they can verify their payment details before submitting the Checkout.

4 Payment Approval

Before the Checkout can be submitted for fulfillment, payment information needs to be validated and certain payment types need to be authorized to make sure there are enough funds. Both validation and authorization are handled by [Payment Approval](#), but your experience does not need to call the endpoint directly. The Checkout API does it for you

when you call [Request a Checkout Submit](#).

TIP: See the [Best Practices](#) section for a sample payment flows.

Notifying Nike of payment after checkout

When customers pay with a deferred payment type, they pay for their order at a third party vendor site after submitting the order for fulfillment. Because the payment event happens outside of the Nike Checkout flow, third party vendors notify Nike of payment events through the [Third Party Payment Notification](#) service.

Payment status changes during fulfillment

After an Order has been submitted for fulfillment, it goes through a series of statuses, some of which involve payment. The Document Order Management System (DOMS) calls the [Fulfillment Payment Notification](#) service to request debits, credits, voids, re-authorizations, and to get payment status.

Listing and Validating Payment Options

List payment options for Checkout

List billing countries for a shipping country

Validate payments

Step 1: List Payment Options for Checkout

Getting a list of valid payment options to display to your customer is typically the first step in adding payment to your experience. The list of valid payment options is calculated based on the customer's [Nike UPMID](#), shopping country, billing country, currency, and items. See the

[Buy Domain Developer's Guide](#) for more information on items in Checkout.

The [Get Payment Options](#) endpoint validates all items passed in the request body. For performance reasons, the products are held in cache for 15 minutes. After the cache expires or if the product is not in cache, the service attempts to get fresh product data from the Merchandised Product API. If the Merchandised Product service is unreachable, the service defaults the product type to "INLINE" and continues validating the product.

Listed below is a sample [Get Payment Options](#) POST request URI. It is not JWT-restricted:

```
https://api.nike.com/paymentoptions/options/v2
```

The results of a successful 200 response lists valid payment methods that a customer can use to pay for the Nike checkout. The list includes the payment name (e.g. "Visa") and payment type (e.g. "CreditCard").

TIPS:

- When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.
- It is a best practice to send all optional request headers, if the data is available, to avoid unexpected responses.
- Even though `items` is an optional request field, it is recommended that you pass it if available so product validation is performed as early as possible in the purchase flow.

Step 2: List Billing Countries for a Shipping Country

You can get the list of valid billing countries based on the customer's shipping country using the [Get Billing Countries for Shipping Country](#) endpoint. Your experience can use this list to display only valid billing countries to the customer and perform shipping/billing country validation as early in the purchase process as possible.

Note that the results are unsorted.

Listed below is a sample [Get Billing Countries for Shipping Country](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/paymentoptions/options/v2/US
```

A successful 200 response lists the billing countries valid for the shipping country path parameter.

TIPS:

- The customer's billing address country must be in the billing country result list in order to purchase.
- When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.
- Your experience needs to pass the country code that the customer is shopping in the `shippingCountry` query parameter.

Step 3: Validate Payments

Use the [Validate Payments](#) endpoint to check each payment on the Checkout selected by the customer is valid for the shipping country. The POST request body should include a client-generated UUID, payment type and billing country for each Checkout payment, and the shipping country.

Listed below is a sample [Validate Payments](#) POST request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/validate_payments/v2
```

A successful 200 response lists all payments provided in the request and true if the billing country and payment type combination is valid, false if not.

TIPS:

- When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.
- It is a best practice to send all optional request headers and body fields, if the data is available, to avoid unexpected responses.

Storing Payment

Add, modify, delete, list stored payments

Modify the default stored payment

Validate a stored credit card

Start a PayPal billing agreement

The Stored Payment service is used to manage (add/update/delete/list) a customer's stored payments. Customers must be registered Nike members and log in to use stored payment. Guest customers are not supported. See [Supported Stored Payment Types](#) to get storage limits by payment type.

Add a new stored payment

Use the [Add Stored Payment](#) endpoint to save a customer's payment for future use. For PayPal stored payments, see [Start a PayPal Billing Agreement](#). The request body varies depending upon the payment type and whether the endpoint is called pre-authorization or post-authorization.

The typical flow for storing a new credit card **pre-authorization** for a customer is:

- 1 Call the [Add Credit Card with CVV](#) endpoint of the Credit Card Submit service to securely transmit credit card information via iFrame to the PCI-certified Credit Card Submit service, passing a client-generated UUID as the `creditCardInfoId`.
- 2 Call [Add Stored Payment](#), passing the same `creditCardInfoId` from **Step 1** to look up the credit card information from short term storage and save it to long term storage.
- 3 Call [Get Stored Payments by UPMID](#) to get all stored payments including the one just saved to confirm that the credit card was securely stored. Credit Card account numbers are masked in the response.

The typical flow for storing a new credit card **post-authorization** for a customer is:

- 1 Call the [Request Checkout Submit](#) endpoint of the Checkout service to validate the Checkout and get payment approval, passing a client-generated UUID as the Checkout id.
- 2 Call [Add Stored Payment](#), passing the `cybersourceRequestId`, `cybersourceRequestToken`, `currency`, and `account` from `paymentToken`, `paymentApprovalId` from **Step 1**.
- 3 Call [Get Stored Payments by UPMID](#) to get all stored payments including the one just saved to confirm that the credit card was securely stored. Credit Card account numbers are masked in the response.

Listed below is the [Add Stored Payment](#) POST request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/customer/savepayment
```

A successful response is a 201.

Modify a credit card stored payment

Use the [Modify Credit Card Stored Payment](#) endpoint to update a stored credit card's expiration month, expiration year, billing address and default payment type flag, or to update a gift certificate default payment type flag. If you only need to update the default payment type flag of either a credit card or gift certificate stored payment, see [Modify the Default Stored Payment](#).

The typical flow for modifying a stored credit card is:

- 1 [Get Stored Payments by UPMID](#) to get all of a customer's stored payments. Credit Card account numbers are masked in the response.

- 2 Call [Modify Credit Card Stored Payment](#) . Get the `payment_id` path parameter from the appropriate `paymentId` field that was returned in the response of **Step 1**.
- 3 [Get Stored Payments by UPMID](#) to get all stored payments including the one just modified to confirm that the credit card was updated. Credit Card account numbers are masked in the response.

Listed below is the [Modify Credit Card Stored Payment](#) PUT request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/customer/storedpayments/  
1686062b-4246-4c3b-ac96-e82a0efae7a0?includebalance=false
```

A successful response is a 202.

Modify the default stored payment

Use the [Modify the Default Stored Payment](#) endpoint to update the default stored payment. This endpoint removes the default flag on the current default stored payment and adds the flag to the stored payment matching the `payment_id` path parameter. An experience can use the default payment type to pre-select a payment method in the shopping flow. If you need to update a stored credit card's expiration month, expiration year, billing address and default payment type flag, see [Modify a credit card stored payment](#).

The typical flow for modifying the default stored payment is:

- 1 [Get Stored Payments by UPMID](#) to get all of a customer's stored payments. Credit Card account numbers are masked in the response.
- 2 Call [Modify Default Stored Payment](#) passing the `payment_id` path parameter using the appropriate `paymentId` field returned in the response of **Step 1**.

3 [Get Stored Payments by UPMID](#) to get all stored payments including the one just modified to confirm that the default credit card was changed. Credit Card account numbers are masked in the response.

Listed below is a [Modify Default Stored Payment](#) PUT request URI. The endpoint is not JWT-restricted.

```
http://api.nike.com/customer/storedpayments/  
7661aea5d-31b6-4ac4-8830-1d61d2c7b043/default
```

A successful response is a 202.

Delete all stored payments

Use the [Delete Stored Payments by UPMID](#) to delete all of a customer's stored payments. If you want to delete just one of a customer's stored payments, see [Delete Stored Payment by ID](#).

The typical flow for deleting all stored payments for a customer is:

- 1 [Get Stored Payments by UPMID](#) to get all of a customer's stored payments. Credit Card account numbers are masked in the response.
- 2 Call [Delete Stored Payments by UPMID](#) to delete all of a customer's stored payments.
- 3 [Get Stored Payments by UPMID](#) to verify that no stored payments are returned.

Listed below is a sample [Delete Stored Payments by UPMID](#) DELETE request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/consumer/storedpayments
```

A successful response is a 204.

Delete Stored Payment by ID

Use the [Delete Stored Payment by ID](#) endpoint to delete a stored payment by payment ID. For example, this endpoint would be called when customers delete a stored payment when managing their payment information in the experience.

The typical flow for deleting a stored payment for a customer is:

- 1 [Get Stored Payments by UPMID](#) to get all of a customer's stored payments. Credit Card account numbers are masked in the response.
- 2 Call [Delete Stored Payment by ID](#) to delete a customer's stored payment for the given `payment_id`.
- 3 [Get Stored Payments by UPMID](#) to verify that no stored payments are returned.

Listed below is a [Delete Stored Payment by ID](#) DELETE request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/consumer/storedpayments/  
37448493-6aea-4a9b-b250-742d3a26c081/?currency=usd
```

A successful response is a 204.

List stored payments

Get Stored Payments by UPMID

Use the [Get Stored Payments by UPMID](#) endpoint to list all of a customer's stored payments. Account numbers are masked in the response. If you are a retail client, use the [Get Stored Payments by UPMID \(Retail\)](#) endpoint instead.

If the request does not contain a shipping address or the shipping address sent does not match a shipping address in a previously placed order, the `validateCVV` field will be set to `true` in the response for Credit Card payment types. This flag indicates that the customer must provide the CVV and it must be validated before the customer can pay for their order using the credit card in the checkout flow.

Listed below is a sample [Get Stored Payments by UPMID](#) POST URI request. This endpoint is not JWT-restricted.

```
https://api.nike.com/consumer/  
storedpayments?currency=USD&includebalance=true&validateshippi  
ng=true
```

A successful 200 response lists all of a customer's stored payments.

Get Stored Payments by UPMID Retail

Use the [Get Stored Payments by UPMID Retail](#) if you are a retail client to list all stored payments for a customer. If you are not a retail client, use the [Get Stored Payments by UPMID](#) endpoint instead.

Listed below is a sample Get Stored Payments by UPMID (Retail). It is not JWT-restricted.

```
https://api.nike.com/consumer/retail_stored_payments/  
v1?currency=USD
```

A successful 200 response lists all of a customer's stored payments except credit cards that have not been used to place a Nike order.

Get Stored Gift Certificate by UPMID

Use the [Get Stored Gift Certificate by ID](#) endpoint to list details for a customer's saved gift certificate.

Listed below is a sample [Get Stored Gift Certificate by ID](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com//consumer/storedpayments/giftcard/  
79847893284923483924?currency=USD
```

A successful 200 response contains gift certificate details with a masked account number.

Get Stored Payment by ID

Use this endpoint to list stored payment details for a `payment_id`. This endpoint is primarily for gift cards but it can be called for any type of stored payment.

When listing a gift card payment type and you don't need the balance, pass `includebalance=false` as a URI parameter for a quicker response. Setting this parameter to false prevents the Stored Payments service from making a balance call to the gift card provider.

Listed below is a [Get Stored Payment by ID](#) GET request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/consumer/storedpayments/  
79847893284923483924
```

A successful 200 response varies depending upon the type of stored payment. A subset of response fields are listed below.

Credit Card

Instead of using credit card number as a lookup key, both CyberSource and Stored Payment use the `paymentToken` generated by CyberSource.

- **type:** always `CreditCard`
- **paymentToken:** Subscription id assigned to this payment by CyberSource
- **cardType:** Type of credit card
- **expiryYear:** Year credit card expires, required for `CreditCard` type
- **expiryMonth:** Month credit card expires, required for `CreditCard` type

Gift Card

- **type:** always `GiftCard`
- **paymentToken:** Random UUID
- **accountNumber:** Unmasked account number
- **balance:** Gift card balance. Not returned if `includebalance` query parameter is `false`
- **pin:** Gift card PIN number

PayPal

- **type:** always `'PayPal'`
- **paymentToken:** PayPal billing agreement id
- **payer:** customer's PayPal email address
- **payerId:** PayPal generated unique id

Deferred Payment

- **type:** Payment type
- **bankName:** Bank name for China payments

Validate Stored Payment Credit Card CVV

Use the [Validate Stored Payment Credit Card CVV](#) endpoint to validate a credit card or gift

certificate based on the request shipping address. If the shipping address does not match a shipping address on a past order or is not sent, the response indicates that the CVV needs to be validated. This endpoint also validates if a credit card is expired. Note that billing address is returned.

Listed below is a sample [Validate Stored Payment Credit Card CVV](#) POST request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/consumer/storedpayments/  
4383642751515000001516?includebalance=false
```

A successful 200 response returns credit card or gift certificate validation and billing information for a `payment_id`.

Start a PayPal Billing Agreement

In order to save PayPal as a stored payment, the customer must [Start a PayPal Billing Agreement](#). This agreement pre-authorizes Nike to charge the customer's PayPal account for purchases without requiring the customer to visit the PayPal site to authorize each purchase. Setting up a PayPal billing agreement makes purchasing easy by keeping the customer in your experience.

Follow these steps to allow customers to Add a PayPal stored payment in your experience.

- 1 Call [Get Stored Payments by UPMID](#) to get a list all of the customer's saved payments to make sure that the customer doesn't already have a PayPal stored payment. Customers can have only one PayPal stored payment.
- 2 Call the [Start a PayPal Billing Agreement](#) endpoint, passing the `returnURL` and `cancelURL` as query parameters so PayPal can return the customer to your experience.
- 3 Redirect the customer to the `redirectURL` in the response so the customer to provide

payment details, approve, and subscribe to the Billing Agreement. Once the customer accepts or cancels the Billing agreement, PayPal redirects the customer to either the returnUrl or cancelURL provided in the query parameter.

- 4 Call the [Add Stored Payment](#) endpoint to save the PayPal billing agreement.
- 5 (Optional) Call [Get Stored Payments by UPMID](#) to list all of the customer's saved payments.

Listed below is a sample [Start a PayPal Billing Agreement](#) GET request URI for the SNKRS mobile experience. This endpoint is not JWT-restricted.

```
https://api.nike.com/commerce/storedpayments/consumer/  
paypalagreement?returnUrl=http://nike.com/SNKRS/  
PayPalAuthenticationSucceeded&cancelUrl=http://nike.com/SNKRS/  
PayPalAuthenticationDidNotSucceed&design=mobile
```

A successful 200 response lists the requestToken, paypalToken, and redirectURL at which the customer can accept the billing agreement.

Credit Card Payment

Add Credit Card with CVV

Add Credit Card without CVV

Add or Update Credit Card CVV

Add or Update Credit Card Expiry and CVV

Validate Credit Card

Store Credit Card for Validation and Purchase

Get Credit Card

Get and Validate Credit Card

This service lists, modifies, deletes and stores a customer's credit card and Apple Pay information. This service accommodates both PCI-certified and non-PCI-certified experiences. Endpoints for non-PCI-certified experiences render an iFrame to collect and retrieve credit card and Apple Pay information. Javascript on the iFrame validates and stores the credit card information once the customer enters it, so your experience doesn't have to call those endpoints. For PCI-certified experiences, it offers endpoints to manage a customer's credit card and Apple Pay information directly.

Add Credit Card with CVV

The [Add Credit Card Info with CVV](#) endpoint is called by experiences that are not [PCI-certified](#). This endpoint renders an iFrame with editable masked credit card number, expiration date, and CVV fields pre-populated with values matching the `creditCardInfoId` passed in the path parameter. If the `creditCardInfoId` is not found, the iFrame renders blank, editable credit card number, expiration date and CVV fields. When each field has a value, the iFrame calls the [Store Credit Card for Validation and Purchase](#) endpoint and temporarily stores the new or updated values. As a last step, the iFrame calls the [Validate Credit Card](#) endpoint to validate the new or updated credit card data.

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Listed below is a sample [Add Credit Card Info with CVV](#) GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services?id=24afd5dc-  
b523-491c-8282-8bed57cd2029&ctx=checkout&language=en
```

The response renders the iFrame below with editable credit card number, expiration date, and CVV fields pre-populated with values looked up based on the `creditCardInfoId` path parameter.

ADD CARD

Card Number	Expiration Date	Security Code
	MM/YY	XXX

Add Credit Card without CVV

The [Add Credit Card Info without CVV](#) endpoint is called by experiences that are not [PCI-certified](#). This endpoint renders an iFrame with the editable masked credit card number and expiration date fields matching the `creditCardInfoId` passed in the path parameter. If the `creditCardInfoId` is not found, the iFrame renders blank, editable credit card number and date fields. When each field has a value, the iFrame calls the [Store Credit Card for Validation and Purchase](#) endpoint and temporarily stores the new or updated values. As a last step, the iFrame calls the [Validate Credit Card](#) endpoint to validate the new or updated credit card data.

TIP: When calling this endpoint through the public router, the **upmid** (for

logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Listed below is a sample [Add Credit Card Info without CVV](#) GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services/add?id=0e13e71d-e952-46af-b3f5-e476befd43ce&language=en
```

The response renders the iFrame below with editable credit card number and expiration date fields pre-populated with values looked up based on the `creditCardInfoId` path parameter.

Number	MM/YY
--------	-------

Add or Update Credit Card CVV

The [Add or Update Credit Card CVV](#) endpoint is called by experiences that are not [PCI-certified](#). This endpoint renders an iFrame with an editable CVV field. When the customer provides a CVV value, the iFrame calls the [Store Credit Card for Validation and Purchase](#) endpoint and temporarily stores the CVV if it found a credit card matching the `creditCardInfoId` path parameter. As a last step, the iFrame calls the [Validate Credit Card](#) endpoint to validate the updated CVV.

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Listed below is a sample [Add or Update Credit Card CVV](#) GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services/cvv?id=0e13e71d-e952-46af-b3f5-e476befd43ce&language=en
```

The response renders the iFrame below with an editable CVV field.

Security Code

XXX

Add or Update Credit Card Expiry and CVV

The [Add or Update Credit Card Expiry and CVV](#) endpoint is called by experiences that are not [PCI-certified](#). This endpoint renders an iFrame with an editable credit card expiration date and CVV fields. When the customer provides the appropriate values, the iFrame calls the [Store Credit Card for Validation and Purchase](#) endpoint and temporarily stores the data for the `creditCardInfoId` path parameter. As a last step, the iFrame calls the [Validate Credit Card](#) endpoint to validate the updated expiration date and CVV values.

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Listed below is a sample [Add or Update Credit Card Expiry and CVV](#) GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services/expcvv?id=0e13e71d-e952-46af-b3f5-e476befd43ce&language=en
```

The response renders the iFrame below with editable expiration date and CVV fields pre-populated with values looked up by the `creditCardInfoId` path parameter.

MM/YY	CVV
-------	-----

Validate Credit Card

The [Validate Credit Card](#) endpoint is typically called by the credit card payment iFrames immediately after the [Store Credit Card for Validation and Purchase](#) endpoint to validate credit card information. The mode query parameter determines which credit card values to validate. It can be called directly by PCI-certified experiences.

- 1: validates expiration date, credit card number and CVV
- 2: validates expiration date and credit card number
- 3: validates CVV
- 4: validates expiration date and CVV

Listed below is a sample [Validate Credit Card](#) GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/creditcardssubmit/24afd5dc-  
b523-491c-8282-8bed57cd2029/isValid?mode=3
```

Each field in the response body is flagged either true or false. True indicates the value is valid; false indicates invalid.

Sample [Validate Credit Card](#) response body for mode=1:

```
{  
  "exp": true,  
  "cc": true,  
  "cvv": true,  
  "isValid": true  
}
```

Sample [Validate Credit Card](#) response body for mode=2:

```
{  
  "exp": true,  
  "cc": true,  
  "isValid": true  
}
```

Sample [Validate Credit Card](#) response body for mode=3:

```
{
```

```
"cvv": true,  
"isValid": true  
}
```

Sample [Validate Credit Card](#) response body for mode=4:

```
{  
  "exp": true,  
  "cvv": true,  
  "isValid": true  
}
```

Store Credit Card for Validation and Purchase

The [Store Credit Card for Validation and Purchase](#) endpoint temporarily stores credit card information for validation and purchase. This endpoint is called by the credit card payment iFrames. It can be called directly by PCI-certified experiences.

Listed below is a sample [Store Credit Card for Validation and Purchase](#) POST request URI. It is not JWT-restricted.

```
https://payment.nike.com/creditcardssubmit/  
bb3360c5-82c3-4d00-b806-a8bf3875555
```

A successful response is a 201.

Get Credit Card

The [Get Credit Card](#) endpoint retrieves masked credit card information for a `creditCardInfoId`. It can be called directly by PCI-certified experiences.

Listed below is a [Get Credit Card](#) GET request URI. It is not JWT-restricted.

```
https://paymentcc.nike.com/creditcardssubmit/  
bb3360c5-82c3-4d00-b806-a8bf3875555
```

A successful 200 response returns credit card information matching the `creditCardInfoId` passed in the path parameter.

Get and Validate Credit Card

The [Get and Validate Credit Card](#) endpoint retrieves masked credit card information and validation status of individual credit card fields for a `creditCardInfoId`. It can be called directly by PCI-certified experiences.

Listed below is a [Get and Validate Credit Card](#) GET request URI. It is not JWT-restricted.

```
https://paymentcc.nike.com/creditcardssubmit/  
bb3360c5-82c3-4d00-b806-a8bf3875555/isValidData?mode=3
```

A successful 200 response returns the `isValid` flag for the credit card as a whole, and `isValid` flags for the credit card number and CVV. It also returns credit card information with a masked credit card number.

Apple Pay Payment

Start an Apple Pay Session

We recommend reading [Apple Pay on the Web](#) and [Apple Pay JS API](#) first.

Step 1: Check that the customer can pay by Apple Pay

To enable paying with Apple Pay on a Safari Web browser, your experience will need to call the [Apple Pay JS API](#) to validate that the Nike customer can pay by Apple Pay on the web and get a `validationURL` to provide merchant identification to Apple.

In order to be eligible to pay by Apple Pay on a Safari web browser, the customer must have:

- access to a Mac and either an iPhone, iWatch, or iPad
- installed the latest macOS Sierra or higher on Mac
- installed iOS 10 or higher on the iPhone, iWatch, or iPad
- set up Apple Pay on the iPhone, iWatch, or iPad
- logged into the same iCloud account on Mac as iPhone, iWatch, or iPad
- installed the latest version of Safari on iPhone, iWatch, or iPad

Step 2: Start an Apple Pay Session

Use the [Start Apple Pay Session](#) endpoint to initialize an Apple Pay session through the Apple gateway. Your experience passes the `validationURL` from **Step 1** to the endpoint and the service will provide the necessary information to Apple Pay to identify Nike as a merchant that accepts Apple Pay payments and start a new Apple Pay session.

Step 3: Store Credit Card for Validation and Purchase

Once you get a successful 200 response from [Start an Apple Pay Session](#), you have all of the information you need to call [Store Credit Card for Validation and Purchase](#) and continue the purchase flow as you would for a credit card.

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Listed below is a sample [Start Apple Pay Session](#) POST request URI and body. The `validationURL` is passed to your experience from the Apple Pay JS API when you [provide merchant validation](#).

```
https://api.nike.com/payment/applepay_sessions/v2
```

```
{
  "validationURL": "https://apple-pay-gateway-pr-
pod1.apple.com/paymentservices/startSession"
}
```

A successful 200 response contains the encrypted signature that your experience needs to pass in the `paymentData` field to [Store Credit Card for Validation and Purchase](#).

[Add credit card steps to complete Apple Pay payment]

Wallet Payment

Start a PayPal Express session

Start a PayPal Mark session

Get PayPal details including shipping and billing addresses stored at PayPal

Your experience can offer two, different PayPal flows, Express and Mark. What's the difference? See the table below.

Express Flow	Mark Flow
Shipping address is stored at PayPal Payment is made in the Nike Experience	Shipping address is stored in the Nike experience Payment is made at the PayPal site

PayPal Express

Follow these steps to implement the PayPal Express payment flow to your experience.

Step 1: Request PayPal Express

Before a customer can pay in the PayPal Express flow, your experience must initiate an Express session at PayPal using the [Request PayPal Express](#) endpoint, passing Checkout information, `returnURL` and `cancelURL`. These URLs are used by PayPal to return the customer to your experience.

This service returns a `paypalToken` and `redirectURL` in the response. When the customer is ready to pay, your experience redirects the customer to the PayPal `redirectURL` passing the `paypalToken`. PayPal uses the token to look up the PayPal session.

At the PayPal site, the customer

- chooses the method of payment such as PayPal balance, debit card, or credit card. All payment methods saved at PayPal have a billing address associated with them.
- chooses an existing or adds a new shipping address
- confirms the payment method and shipping address

OR

- cancels the PayPal Express session

When the customer confirms the payment method and shipping address on the PayPal site, PayPal redirects the customer to the `returnURL` passed in the request body. The `returnURL` is typically to a Checkout review page in your experience from which the customer can choose to submit the Checkout for fulfillment.

If the customer cancels the PayPal Express session on the PayPal site, PayPal redirects the customer to the `cancelURL` passed in the request body. The `cancelURL` is typically to a billing page in your experience where the customer can choose an alternate payment method.

TIP: The `returnURL` you provide in the [Request PayPal Express request body](#) is a link to your experience to which PayPal will redirect the customer after confirming the payment method and shipping address. The `returnURL` provided in the [Request PayPal Express response body](#) is a link to the PayPal site to which your experience will redirect the customer to select a shipping address and payment method.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#)

guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample [Request PayPal Express](#) PUT request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/paypal_express/v1
```

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling eta. A successful response in `COMPLETED` status also includes the response object containing the job results.

Step 2: Retrieve PayPal Express Job

Use the [Retrieve PayPal Express Job](#) endpoint to check the status of the [Request PayPal Express](#) job. After receiving a HTTP 202 and waiting the duration of the eta time, call the endpoint using the same UUID to check the status of your job. If the status is not `COMPLETED`, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the Request PayPal Express response.

Listed below is a sample [Retrieve PayPal Express Job](#) GET request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/paypal_express/v1/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe783424
```

A successful 200 response lists the `paypalToken` and `redirectURL`

Step 3: Request PayPal Details

Use the [Request PayPal Details](#) endpoint to retrieve and validate PayPal data, including shipping and billing addresses stored at PayPal. You will need to pass the `paypalToken` and `shoppingCountry` in the request body returned in either the [Request PayPal Mark](#) or [Request PayPal Express](#) response.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve

the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#) guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample [Request PayPal Details](#) POST request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/payment/paypal_details/v1
```

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling eta. A successful response in `COMPLETED` status also includes the response object containing the job results.

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Step 4: Retrieve PayPal Details Job

Use the [Retrieve PayPal Details Job](#) endpoint to check the status of the [Request PayPal Details](#) job. After receiving a HTTP 202 and waiting the duration of the eta time, call this endpoint using the same UUID to check the status of your job. If the status is not `COMPLETED`, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the *PayPal Details* response.

Listed below is a sample [Retrieve PayPal Details Job](#) GET request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/paypal_details/v1/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe78
```

A successful 200 response lists shipping and billing addresses.

PayPal Mark

Follow these steps to implement the PayPal Mark payment flow to your experience.

Step 1: Request PayPal Mark

Before a customer can pay in the PayPal Mark flow, your experience must initiate a Mark session at PayPal using the [Request PayPal Mark](#) endpoint, passing Checkout information,

shipping address, `returnURL` and `cancelURL`. These URLs are used by PayPal to return the customer to your experience.

This endpoint starts a new PayPal Mark session and passes the Checkout information and shipping address to PayPal for session storage. PayPal generates a `paypalToken` and `redirectURL` and the endpoint returns them in the response. When the customer is ready to Pay, your experience redirects the customer to the PayPal `redirectURL` passing the `paypalToken`. PayPal uses the token to look up the PayPal session.

At the PayPal site, the customer

- chooses the method of payment such as PayPal balance, debit card, or credit card. All payment methods saved at PayPal have a billing address associated with them.
- confirms payment and pays for the Checkout

OR

- cancels the PayPal Mark session

After the customer pays for the Checkout or cancels the PayPal Mark session on the PayPal site, PayPal redirects the customer to either the `returnURL` or `cancelURL` passed in the request body. The `returnURL` is typically to an order confirmation page in your experience.

TIP: The `returnURL` you provide in the [Request PayPal Mark request body](#) is a link to your experience to which PayPal will redirect after the customer pays for their Checkout at the PayPal site. The `returnURL` provided in the [Request PayPal Mark response body](#) is a link to PayPal to which your experience will redirect the customer to pay.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve

the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#) guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample [Request PayPal Mark](#) POST request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/paypal_mark/v1
```

TIP:When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling eta. A successful response in `COMPLETED` status also includes the response object containing the job results.

Step 2: Retrieve PayPal Mark Job

Use the [Retreive PayPal Mark Job](#) endpoint to check the status of the [Request PayPal Mark](#) job. After receiving a HTTP 202 and waiting the duration of the eta time, call this endpoint using the same UUID to check the status of your job. If the status is not `COMPLETED`, continue the cycle of waiting the eta period and checking the job status.

Endpoint Details

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started

- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the PayPal Mark response.

Listed below is a sample [Retrieve PayPal Mark Job](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/paypal_mark/v1/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe78
```

A successful 200 response lists the `paypalToken` and `redirectURL`.

Step 3: Request PayPal Details (PayPal Mark)

In order to display the billing address stored at PayPal to the consumer, you will need to call the [Request PayPal Details](#) endpoint. See [Request PayPal Details](#) in the PayPal Express section for more information.

Step 4: Retrieve PayPal Job (PayPal Mark)

For details on how to [Retrieve PayPal Details Job](#) see [Retrieve PayPal Details Job](#) in the PayPal Express section for more information.

Deferred Payment

Generate a signed link to pay at third party vendor sites such as iDeal, Sofort, Alipay, Tenpay and UnionPay

Generate a signed link to pay by WeChat

Get the status of a deferred payment

Step 1: Request Deferred Payment Form

Use the [Request Deferred Payment Form](#) endpoint to generate a signed link used to redirect the customer to pay at a third-party site or app. This endpoint is used for experiences that support iDeal, Sofort and/or Alipay, Tenpay, and UnionPay China payment types. For WeChat payment, see the [Request WeChat Deferred Payment](#) endpoint.

In the request body, your experience will need to pass the `approvalId` returned from [Request Payment Approval](#) and your experience's `returnURL` that the third party vendor will redirect the consumer to after making payment at their site.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#) guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample [Request Deferred Payment Form](#) POST request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/deferred_payment_forms/v1
```

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling eta. A successful response in `COMPLETED` status also includes the response object containing the job results.

TIPS:

- When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.
- Parsing the 'COMPLETED' job result directly is a best practice because it eliminates making another service call.

Step 2: Retrieve Deferred Payment Form Job

Use the [Retrieve Deferred Payment Form Job](#) endpoint to check the status of the [Request Deferred Payment Form](#) job. After receiving a HTTP 202 and waiting the duration of the eta time, call this endpoint using the same UUID to check the status of your job. If the status is not `COMPLETED`, continue the cycle of waiting the eta period and checking the job status. In the case of checking the status of [Request WeChat Deferred Payment](#), see the [Request WeChat Deferred Payment](#) section.

To know if the job is done, check the value of the `status` field in the response body as follows:

- "status": "PENDING": job processing has not started
- "status": "IN_PROGRESS": job processing in progress
- "status": "COMPLETED": job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the Deferred Payment Form response.

Listed below is a sample [Retrieve Deferred Payment Form Job](#) GET request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/deferred_payment_forms/v1/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe78
```

A successful 200 response in COMPLETED status contains the signed third party vendor URL at which the consumer can pay for their Nike order.

Step 3: Redirect the consumer to the third party payment site

Use the values from the [Request Deferred Payment Form](#) job to redirect the consumer to

pay at the third party site. Depending upon the vendor and the experience the consumer is shopping in, the response may contain a URL to generate a QR code for the deferred payment page or a form action URL and HTTP method.

Step 4: Request Deferred Payment Status

Use the [Request Deferred Payment Status](#) endpoint to check the status of a deferred payment with the third-party Vendor.

Your experience will need to pass the `approvalId` returned from [Request Payment Approval](#) and any `vendorData` returned in the [Request Deferred Payment Form](#) response.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#) guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample [Request Deferred Payment Status](#) POST request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/payment/deferred_payment_status/v1
```

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

A successful 202 response in `COMPLETED` status lists the payment status and amount paid if the payment status is `PAYMENT_SUCCESSFUL`. If the 202 response is `PENDING` or `IN_PROGRESS`, it includes a link to the job and a status polling eta.

Step 5: Retrieve Deferred Payment Status Job

Use the [Retrieve Deferred Payment Status Job](#) endpoint to check the status of the [Request Deferred Payment Status](#). After receiving a HTTP 202 and waiting the duration of the eta time, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the Deferred Payment Status response.

Listed below is a sample [Retrieve Deferred Payment Status Job](#) GET request URI. This endpoint is not JWT-restricted.


```
https://api.nike.com/payment/deferred_payment_status/v1/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe78
```

A successful 200 response in COMPLETED status lists the payment status and amount paid if the payment status is PAYMENT_SUCCESSFUL.

WeChat Deferred Payment

We recommend reading [JSAPI WeChat Browser](#) and [WeChat Documentation](#) first.

WeChat Desktop Flow

Consumers scan a QR code to pay with WeChat in a web browser flow, also known as native payment.

The Vendor generates a transaction QR Code according to the WeChat Payment Protocol and the Payer goes to “Scan QR Code” in their WeChat in order to complete payment. This mode is applicable to payments made on websites, physical stores, media advertising, or other scenarios.

Desktop flow: Experience displays QR code at the end of checkout (what generates the QR code?). Consumer scans code with phone and opens WeChat app. User pays. (when is wechat deferred payment called? how is nike notified of payment?)

Step 1: Generate a QR Code

If the customer is shopping in a desktop web experience and chooses to pay by WeChat, your experience will need to generate a QR code at the end of Checkout for the customer to scan on a mobile phone. Generate the QR code by following the steps in WeChat’s documentation on [WeChat Login for WebApps](#).

Step 2: Request WeChat Deferred Payment

Use the [Request WeChat Deferred Payment](#) endpoint to generate the necessary values to initiate a session in the WeChat Pay Browser Phone App from a mobile or desktop web browser. For other third party deferred payment types, see [Request Deferred Payment Form](#).

The mobile web flow opens the WeChat Payment app directly when it is time to pay for the Nike Checkout.

The Desktop WeChat flow generates a QR code. Nike customers use their Mobile phone to scan the code to open the WeChat Payment App on their mobile device.

Using the code returned from WeChat in **Step 1** and the `approvalId` returned in the [Request Payment Approval](#) response, call [Request WeChat Deferred Payment](#).

Listed below is a sample [Request WeChat Deferred Payment](#) POST request URI. This endpoint is not JWT-protected.

```
https://api.nike.com/payment/deferred_wechat_payments/v1
```

A successful 202 response includes a link to the job and a status polling eta.

TIP: When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Step 3: Retrieve WeChat Deferred Payment Job

Use the [Retrieve WeChat Deferred Payment Job](#) endpoint to check the status of the [Request](#)

[WeChat Deferred Payment](#) job. After receiving a HTTP 202 and waiting the duration of the eta time, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the Deferred Payment Form response.

Listed below is a sample [Retrieve Deferred WeChat Payment Job](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/deferred_wechat_payments/v1/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe78
```

Response Body

The HTTP 200 response from *Deferred WeChat Payment Job* contains information about how to retrieve the results of your job via the 'Deferred WeChat Payment Job' endpoint. The response body fields are the same as those returned in the [Deferred Payment WeChat](#) response except the resourceType is payment/deferred_wechat_payments/jobs.

WeChat Mobile Flow

When consumers pay by WeChat in a mobile phone web browser, your experience will open the WeChat app to process the payment, also known as In-App payment.

In-App payment refers to a mobile-based payment in which the Vendor calls the WeChat payment module by using the open SDK integrated in their mobile-based app to pay for transactions.

After consumer chooses to pay with wechat, experience calls WeChat deferred payment to get values. Calls JS API using those values to open WeChat App. User pays. WeChat sends callback to notify experience of payment. Experience loads order confirmation page.

Payment Preview

Preview the allocation of payment amounts across one or more payment methods selected by the customer

Step 1: Request a Payment Preview

Nike customers can pay by one or more gift cards and vouchers and another payment type such as PayPal or credit card. The [Request Payment Preview](#) endpoint allocates payment to the gift cards or voucher with the highest balance first and then to the rest of the gift cards and vouchers on the Checkout in ascending balance order. If the total balance of all gift cards and vouchers is less than the order amount, the service allocates the balance of the order to a second payment type.

The `paymentPreviewId` returned by this service is a required key when calling [Request Checkout Submit](#) in the BUY API to validate and authorize/debit payment before submitting a Checkout to Nike for fulfillment.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#) guide to learn more about working with asynchronous Nike APIs.

TIP:

- When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.
- `Promotion` in the `response.payments.type` field is an indicator that the entire order is allocated to a promotion.

A few notes about the Payment Preview request body:

- **checkoutId** is a [UUID](#) that payments are associated to. It is generated by [Request Checkout Preview](#).
- `items.shippingAddress.county` holds the shipping address county for the US. Outside of the US, it holds regional data and is required in CN and JP.
- **paymentInfo** is an array of payment types for the Checkout and is required except for the [PayPal Mark](#) flow.

- `paymentInfo.billingInfo` is required for all payment methods except [PayPal](#).
- `PaymentInfo.creditCardInfoId` is a required field when paying by credit card that is not a [stored payment](#). It is a PCI-required token used to look up credit card information and is generated by the [Store Credit Card for Validation and Purchase](#) endpoint. If the credit card is a stored payment, then `PaymentInfo.paymentId` is required.

Listed below is a sample [Request Payment Preview](#) POST request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/payment/preview/v2
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Check if the Payment Preview job is finished

Use the [Retrieve Payment Preview Job](#) endpoint to check the status of the [Request Payment Preview](#) job after receiving a HTTP 202 and waiting the duration of the eta time. If the status is not COMPLETED, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the `status` field in the response body as follows:

- `"status": "PENDING"`: job processing has not started
- `"status": "IN_PROGRESS"`: job processing in progress
- `"status": "COMPLETED"`: job has completed

Once you receive a job status of "COMPLETED", get the results of your job by parsing the data in the `response` object from this endpoint. Alternatively, follow the link to the [Retrieve Payment Preview Result](#) endpoint which is provided in the `links` object response body.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the `id` field returned in the *Request Payment Preview* response.

Parsing the “COMPLETED” job result directly is a best practice because it eliminates making another service call.

Listed below is a sample [Retrieve Payment Preview Job](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/preview/v2/jobs/308830db-  
bcca-45a6-8d81-20f3b6dafd9e
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Payment Preview job results.

Step 3: Retrieve Payment Preview Result

After calling the [Request Payment Preview](#) to start the job and [Retrieve Payment Preview Job](#) to check that the status of the job is “COMPLETED”, you can optionally call the [Retrieve Payment Preview Result](#) endpoint to retrieve the job result. It is optional because the result is also returned in the [Retrieve Payment Preview Job](#) when it is in “COMPLETED” status.

Listed below is a sample [Retrieve Payment Preview Result](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/preview_results/v2/308830db-  
bcca-45a6-8d81-20f3b6dafd9e
```

A successful 200 response lists the payment types on the Checkout and the amount allocated to each type.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Get the {id} path parameter from the id job UUID in the *Request Payment Preview* response.

Payment Approval

Perform fraud check, validation, and authorization/debit for all payment types on a customer's Checkout

Void a Payment Approval request

Get a Payment Approval summary

TIP: Except for the [Get Payment Approval Summary](#) endpoint, only service-to-service calls should be made to the Payment Approval endpoints. [Request Checkout Submit](#) calls [Request Payment Approval \(POST\)](#) as a last step in the Checkout flow to validate and authorize/debit payment before submitting a Checkout to Nike for fulfillment. **An experience should not call this service directly.**

Step 1: Request Payment Approval

Both the [Request Payment Approval \(POST\)](#) and [Request Payment Approval \(PUT\)](#) endpoints perform fraud check, validation, and authorization/debit for all payment types on a customer's Checkout. The PUT version of the endpoint requires a `paymentApprovalId` path parameter. This is helpful if the [Request Payment Approval](#) response times out. In that case, [Request Payment Void](#) would need to be called with the same `paymentApprovalId` to reverse the original Payment Approval request.

[Request Payment Approval \(POST\)](#) and [Request Payment Approval \(PUT\)](#) must be called after [Request Checkout Preview](#) so that order allocation is calculated and the `paymentPreviewId` is assigned.

The Payment Approval service validates the payment allocation performed by the [Payment Preview](#) service, recalculating if necessary, and evaluates that the selected payment methods and items on Checkout are valid. If one or more payment type validations fail, all gift card debits and all credit card and PayPal authorizations are rolled back. This service uses the `paymentPreviewId` to look up the Checkout payment methods so it does not require payment information be passed in the request.

This endpoint operates **asynchronously**, which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the [Using NDe APIs](#)

guide to learn more about working with asynchronous Nike APIs.

Listed below is a [Request Payment Approval](#) POST request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/payment/approval/v2
```

Listed below is a sample [Request Payment Approval](#) PUT request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/payment/approval/v2/  
2722be3a-0341-11e6-b512-3e1d05defe783424
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Retrieve Payment Approval Job

Use the [Retrieve Payment Approval Job](#) endpoint to check the status of the [Request Payment Approval](#) (POST) or [Request Payment Approval](#) (PUT) job. After receiving a HTTP 202 and waiting the duration of the eta time specified in the response, call [Retrieve Payment Approval Job](#) using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the

data in the response object from this endpoint.

Listed below is a sample [Retrieve Payment Approval Job](#) GET request. It is not JWT-restricted.

```
https://api.nike.com/payment/approval/v2/jobs/  
2722be3a-0341-11e6-b512-3e1d05defe783424
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Payment Approval job results.

Step 3: Retrieve Payment Approval Result (Optional)

After calling [Request Payment Approval](#) (POST) or [Request Payment Approval](#) (PUT) to start the job and [Retrieve Payment Approval Job](#) to verify the status of the job is “COMPLETED”, you can optionally call [Retrieve Payment Approval Results](#) to retrieve the result of your Payment Approval job. This step is optional because the Payment Approval result is also returned in the [Retrieve Payment Approval Job](#) when it is in COMPLETED status. DOMS is currently the only service that calls this endpoint.

Listed below is a sample [Retrieve Payment Approval Results](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/approval_results/v2/  
ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response lists the Payment Approval job results.

Step 4: Request Payment Void (Optional)

Use the [Request Payment Void](#) endpoint to void a [Request Payment Approval](#). This endpoint

should be called when the original [Request Payment Approval](#) (PUT) timed out using the same `id` sent in the path parameter. If a credit card or PayPal was used in the original Payment Approval request, this endpoint reverses the authorization. If a gift card was used in the original Payment Approval request, it reverses the debit.

Listed below is a sample [Request Payment Void](#) DELETE request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/payment/approval_results/v2/  
ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful response is a 204.

Step 5: Get Payment Approval Summary (Optional)

TIP: This endpoint can be called directly by an experience to display the masked results of a Request Payment Approval in `COMPLETED` status.

Use the [Get Payment Approval Summary](#) endpoint to retrieve a summary of a successful Payment Approval request. The results are available for 30 minutes after the original Payment Approval request was made. This endpoint can be called by an experience because sensitive account information is masked. The results can be used to display payment approval results confirming a consumer's order.

Note that if the Payment Approval result is not in either `ACCEPT` or `PENDING_PAYMENT` status, the service returns a 404 response.

TIPS:

When calling this endpoint through the public router, the **upmid** (for logged in customers), **appld** and **usertype** headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by Nike Unite.

Listed below is a sample [Get Payment Approval Summary](#) GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/approval_summary/v1/  
ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response lists a summary of a successful payment approval with masked account information.

Fulfillment Payment Notification

The following payment actions can be taken on an order after it has been submitted for fulfillment.

Debit an account

Credit an account

Void a payment authorization

Reauthorize a payment

Get payment status

Nike's Distributed Order Management System (DOMS) is currently the only consumer of this service.

All Payment Gateway endpoints are **asynchronous** so after making the initial Payment Notification request, you will need to poll the appropriate job endpoint to get the results.

TIP: The `id` path parameter for PUT requests to this service is a client-supplied value used to look up the job result.

Debit an account

Step 1: Submit the debit request

Use the [Request Debit](#) endpoint to debit funds for orders paid with credit card, PayPal, Gift Certificate, and Klarna payment types. This endpoint is called when it is time to transfer funds from the customer's account to Nike's account, such as when the customer's shipment leaves the warehouse.

Listed below is a sample [Request Debit](#) PUT request URI. **This endpoint is JWT-restricted.**

```
/payment/debits/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

TIP: The `authorizationRequestId` and `authorizationRequestToken` request body fields come from the `requestId` and `requestToken` in the [Request Payment Approval](#)

response.

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Check the debit job status

Use the [Retrieve Debit Job](#) endpoint to retrieve the results of the [Request Debit](#) job. After receiving a HTTP 202 and waiting the duration of the eta time specified in the response, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the **status** field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample [Retrieve Debit Job](#) GET request URI. This endpoint is not JWT-restricted.

```
/payment/debits/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Debit job results.

Credit an account

Step 1: Submit the credit request

When a Nike customer returns one or more products, call the [Request Credit](#) endpoint to return the money to the same account the customer used to pay for the product. You can issue a full credit (also known as a refund) for the full charge amount, or you can issue multiple, partial credits up to the full charge amount. If you try to credit more than the charge amount, you will receive an error.

TIP: You can get the `debitRequestId` and `debitRequestToken` values to pass in the credit request body from the `requestId` and `requestToken` fields in the [Retrieve Debit Job](#).

Listed below is a sample [Request Credit](#) PUT request URI. **This endpoint is JWT-restricted**

```
/payment/credits/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Check credit job status

After calling [Request Credit](#) and receiving a HTTP 202 response, execute a request to [Retrieve Credit Job](#) using the same Credit ID to check the status of your job.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample [Retrieve Credit Job](#) GET request URI. This endpoint is not JWT-restricted.

```
/payment/credits/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Credit job results.

Void an authorization

Step 1: Request an unauth

Use the [Request Unauth](#) endpoint to release the hold on funds set aside by authorization for a future debit. See the [Payment Gateway API](#) for details on void request and response information for each payment type.

TIP: The `authorizationRequestId` and `authorizationRequestToken` request body fields come from the `requestId` and `requestToken` in the [Request Payment Approval](#) response.

Listed below is a sample [Request Unauth](#) PUT request URI. **This endpoint is JWT-**

restricted

```
/payment/authorize_reversals/v1/ae6575a7-8c0e-44ef-  
b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Retrieve the Unauth Job

Use the [Retrieve Unauth Job](#) endpoint to retrieve the status of the [Request Unauth](#) job. After receiving a HTTP 202 and waiting the duration of the eta time specified in the response, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the eta period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample [Retrieve Unauth Job](#) GET request URI. This endpoint is not JWT-restricted.

```
/payment/authorize_reversals/v1/jobs/ae6575a7-8c0e-44ef-  
b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Unauth job results.

Reauthorize a payment

Step 1: Submit the reauthorization request

Authorizations are only valid for a certain time period. When an authorization expires, you will need to call [Request Reauthorization](#) to ensure there are still enough funds to pay for the order. One reason for auth expiration is when there is a delay at the warehouse. In this case, the product cannot be shipped and paid for before the auth expires.

TIP: You can get the `originalRequestId` and `originalRequestToken` request body fields come from the `requestId` and `requestToken` in the [Request Payment Approval](#) response.

Listed below is a sample [Request Reauthorization](#) PUT request URI. **This endpoint is JWT-restricted.**

```
/payment/reauthorizes/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Check the reauthorization job status

Use the [Retrieve Reauthorization job](#) to check the status of the [Request Reauthorization](#) job.

To know if the job is done, check the value of the status field in the response body as

follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample [Retrieve Reauthorization job](#) GET request URI. It is not JWT-restricted.

```
/payment/reauthorizes/v1/jobs/ae6575a7-8c0e-44ef-  
b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Reauthorization job results.

Creating a Gift Certificate

Step 1: Submit payment request for certificate

You can create a gift certificate for a customer by calling the [Request Certificate](#) endpoint. You specify the currency, amount and email address for the recipient and the service returns a gift card number and pin set to the amount and currency passed in the request.

Listed below is a sample [Request Certificate](#) PUT request URI. **This endpoint is JWT-restricted.**

```
/payment/create_certificates/v1/ae6575a7-8c0e-44ef-  
b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Check the Request Certificate job status

After calling [Request Certificate](#) and receiving a HTTP 202 response, execute a request to [Retrieve Certificate Job](#) using the same id to check the status of your job.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample [Retrieve Certificate Job](#) GET request URI. This endpoint is not JWT-restricted.

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Certificate job results.

Query Payment Status

Step 1: Submit the payment status query

Use the [Request Payment Status Query](#) to check the payment status of an order. Before cancelling an order paid by a deferred payment type such as WeChat or Alipay, you will need to know if the order has been PAID so you can [Request Credit](#) to return the funds to the consumer's account.

TIP: The `requestId` and `requestToken` request body fields come from the `requestId` and `requestToken` in the [Request Payment Approval](#) response.

Listed below is a sample [Request Payment Status Query](#) PUT request URI. **This endpoint is JWT-restricted.**

```
/payment/queries/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling eta.

Step 2: Check the Retrieve Payment Status Query job status

After calling [Request Payment Status Query](#) and receiving a HTTP 202 response, execute a request to [Retrieve Payment Status Query](#) using the same `id` to check the status of your job.

To know if the job is done, check the value of the `status` field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample [Retrieve Payment Status Query](#) GET request URI. This endpoint is not JWT-restricted.

```
/payment/queries/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Payment Status job results.

Third Party Payment Notification

Third party payment vendors call the [Payment Notification API](#) to notify Nike of payment status changes for an order with an offline payment type.

When customers choose to pay for their Nike order with an offline payment type, customers pay after the order is submitted for fulfillment. The customer must pay within a certain time period defined by the vendor or Nike automatically cancels the order. After the customer pays the third party vendor, the vendor calls the Payment Notification service to notify Nike of payment. This service handles updating the payment status and making sure that DOMS is notified of the payment event so it can update the order status.

The supported third party payment vendors are:

- Alipay

- Konbini
- Sofort
- Unionpay
- WeChat

Each vendor has its own synchronous endpoint with the POST body defined by the vendor. The service response is either `success`, or `failure` and reason. See the [Payment Notification API](#) for details on request and response by vendor.

API Quick Reference

Payment ApplePay

- [Start Apple Pay Payment Session](#)

Payment Approval

- [Request Payment Approval \(POST\)](#)
- [Request Payment Approval \(PUT\)](#)
- [Retrieve Payment Approval Job](#)
- [Retrieve Payment Approval Results](#)
- [Request Payment Void](#)
- [Get Payment Approval Summary](#)

Credit Card Submit

- [Add Credit Card with CVV](#)
- [Add Credit Card without CVV](#)
- [Add or Update Credit Card CVV](#)
- [Add or Update Credit Card Expiry and CVV](#)
- [Validate Credit Card](#)
- [Store Credit Card for Validation and Purchase](#)
- [Get Credit Card \(with validation\)](#)

- [Get and Validate Credit Card](#) (without validation)

Deferred Payment

- [Request Deferred Payment Form](#)
- [Retrieve Deferred Payment Form Job](#)
- [Request Deferred Payment Status](#)
- [Retrieve Deferred Payment Status Job](#)
- [Request WeChat Deferred Payment](#)
- [Retrieve WeChat Deferred Payment Job](#)

Payment Gateway

- [Request Debit](#)
- [Retrieve Debit Job](#)
- [Request Credit](#)
- [Retrieve Credit Job](#)
- [Request Unauth](#)
- [Retrieve Unauth Job](#)
- [Request Reauthorization](#)
- [Retrieve Reauthorization Job](#)
- [Request Payment Status Query](#)
- [Retrieve Payment Status Query](#)
- [Request Certificate](#)
- [Retrieve Certificate Job](#)

Payment Options

- [Get Payment Options](#)
- [Get Billing Countries for Shipping Country](#)
- [Validate Payments](#)

Payment Preview

- [Request Payment Preview](#)
- [Retrieve Payment Preview Job](#)
- [Retrieve Payment Results](#)

Stored Payment

- [Get Stored Payments by UPMID](#)
- [Get Stored Payments by UPMID \(Retail\)](#)
- [Get Stored Gift Certificates by ID](#)
- [Add Stored Payment](#)
- [Delete Stored Payments by UPMID](#)
- [Modify Default Stored Payment](#)
- [Modify Credit Card Stored Payment](#)
- [Get Stored Payment by ID](#)
- [Delete Stored Payment by ID](#)
- [Validate Stored Payment Credit Card CVV](#)
- [Start a PayPal Billing Agreement](#)

Payment Wallet

- [Request Paypal Details](#)
- [Retrieve Paypal Details JobD](#)
- [Request Paypal Express](#)
- [Retrieve Paypal Express Job](#)
- [Request Paypal Mark](#)
- [Retrieve Paypal Mark Job](#)

Caching Data

The Payment API makes use of data caching to optimize service SLAs. The first time data is fetched or when the cache expires, the Payment service makes a call to get the latest data and adds it to the cache.

The PaymentWallet, PaymentPreview, PaymentApproval and StoredPayments services handle gift card balances. Retrieving the balance of a gift card requires a call to a third-party gift card provider, which can slow down the Payment service's response, especially in high volume traffic. To avoid this scenario, the private gift card Service, which is responsible for retrieving gift card data and is called by the PaymentWallet, PaymentPreview, PaymentApproval and StoredPayments services, caches the gift card balance after retrieval. The cache time varies based on the balance. If the gift card has a positive balance, the gift card service caches the balance for 5 minutes; If the gift card has a 0 balance, the gift card service caches the balance for 30 minutes.

The PaymentOptions, PaymentWallet, PaymentPreview and PaymentApproval services use product and SKU data as part of validation. For performance reasons, these services cache product and SKU data for 30 minutes in order to reduce the amount of calls to the [Merchandised Products API](#) to get the latest data.

Best Practices

Sample Flows

Payment API flows vary based on the payment method and the user experience. Listed below are a few examples of Payment API usage.

Sample Credit Card Payment Flow without Stored Payment

Listed below is a sample credit card payment flow. In this flow, the customer is a Nike registered member who has added products and services to the [Checkout](#), provided a shipping address, and has the intention to purchase.

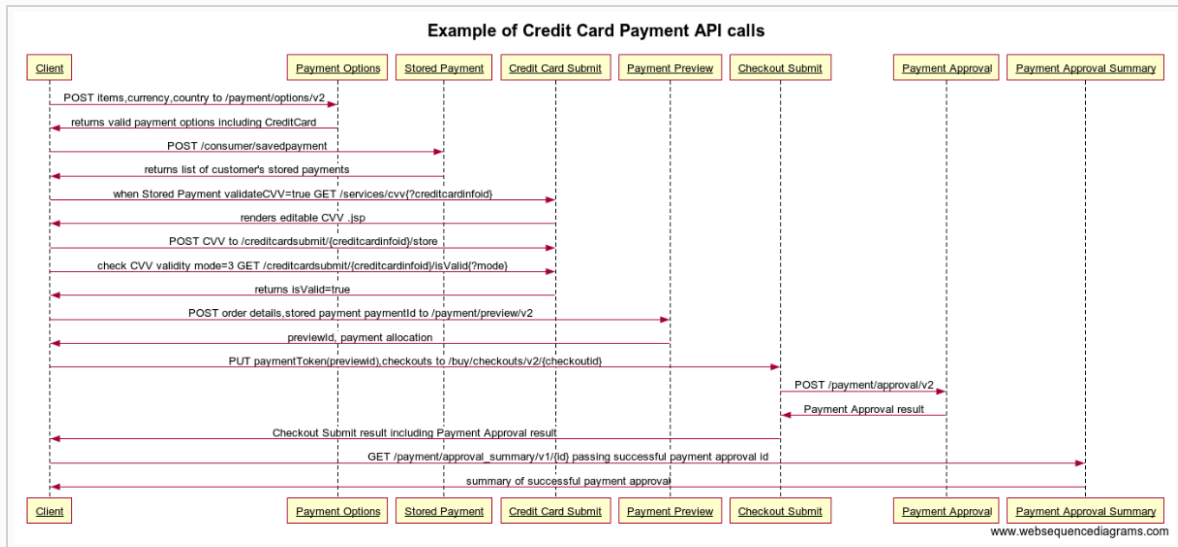
- 1 Your experience calls [Payment Options](#) to get a list of valid payment methods for the customer.
- 2 The customer selects to pay by a non-stored credit card from the list of payment options

in your app.

- 3 Your experience calls [Credit Card Submit](#) to capture the customer's credit card information in a PCI-compliant UI. Since the customer is a registered member, your experience may ask to store her credit card for future use.
- 4 If the registered member has chosen to store the credit card for reuse, your experience calls [Stored Payment](#) to validate, securely store, and display masked credit card information.
- 5 Your experience passes the Checkout and credit card information to [Payment Preview](#) in order to allocate the order total across the selected payment methods and generate the Payment Preview ID.
- 6 Your experience calls [Checkout Preview](#) to validate Checkout and calculate item pricing, shipping and taxes.
- 7 Your experience calls [Checkout Submit](#) with the Payment Preview ID to validate payment for a final time, authorizes the credit card by calling [Payment Approval](#), and submits the Checkout for fulfillment.
- 8 Your experience calls [Payment Approval Summary](#) to display the payment details to the customer for order confirmation.

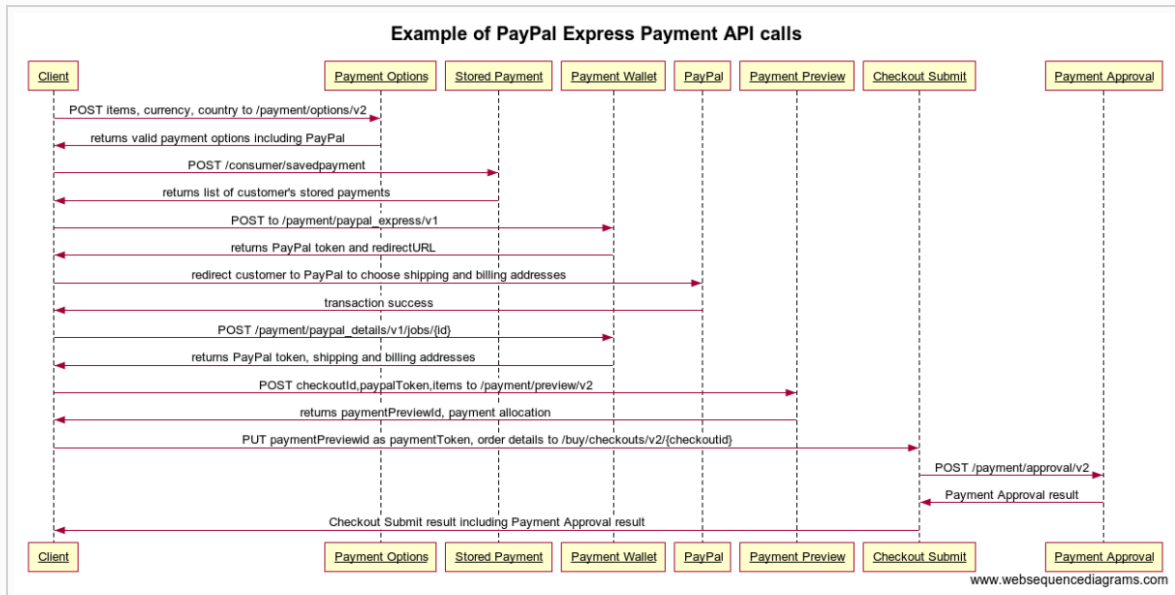
Sample Credit Card Payment Flow with Stored Payment

In this Payment flow, the customer chooses to pay by Credit Card that is saved as a stored payment method. The customer must provide the CVV for validation because the shipping address passed into the call is either new or different from previous shipping addresses on past orders.



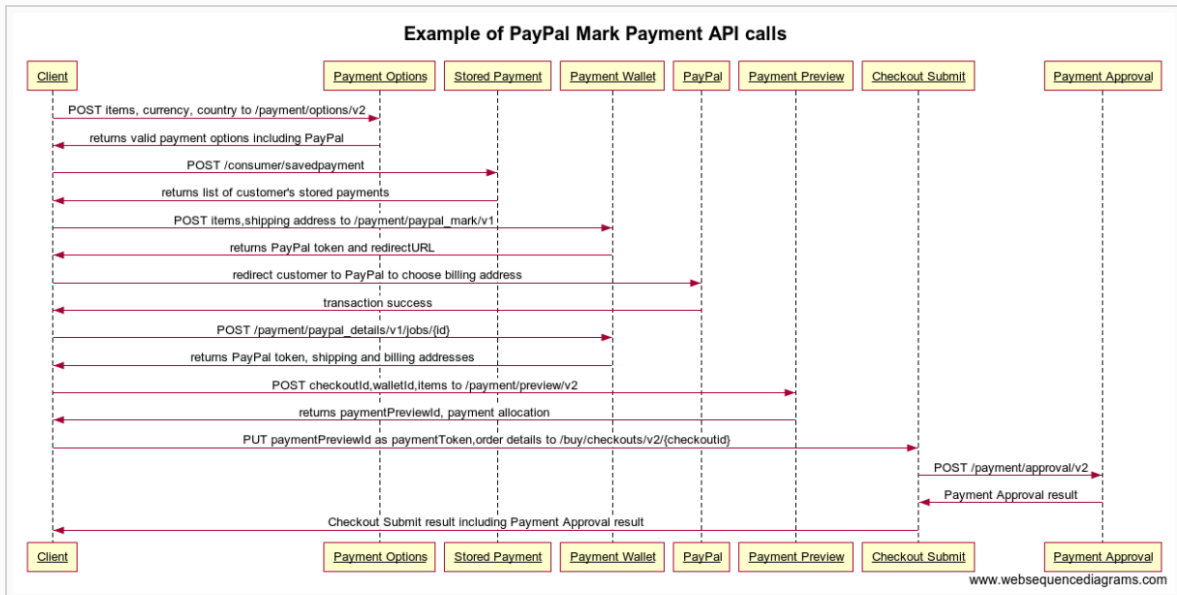
Sample PayPal Express flow

In this flow, the customer is redirected to the PayPal site after choosing to pay by PayPal Express in the Nike experience. The customer selects the shipping and billing addresses on the PayPal site. Based on the PayPal token, the Payment Wallet service returns the shipping and billing addresses from PayPal for display on the order confirmation.



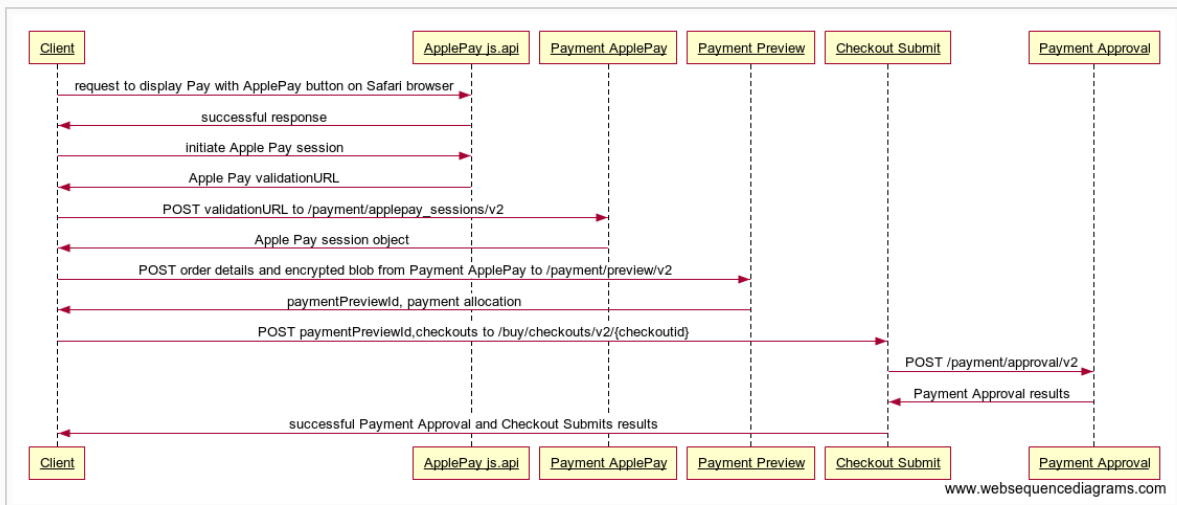
Sample PayPal Mark flow

In this flow, the customer chooses to pay by PayPal Mark and provides the shipping address in the Nike experience. From order review, the customer is redirected to the PayPal site to select the billing address and pay. Based on the PayPal token, the Payment Wallet service returns the shipping and billing addresses from PayPal for display on the order confirmation.



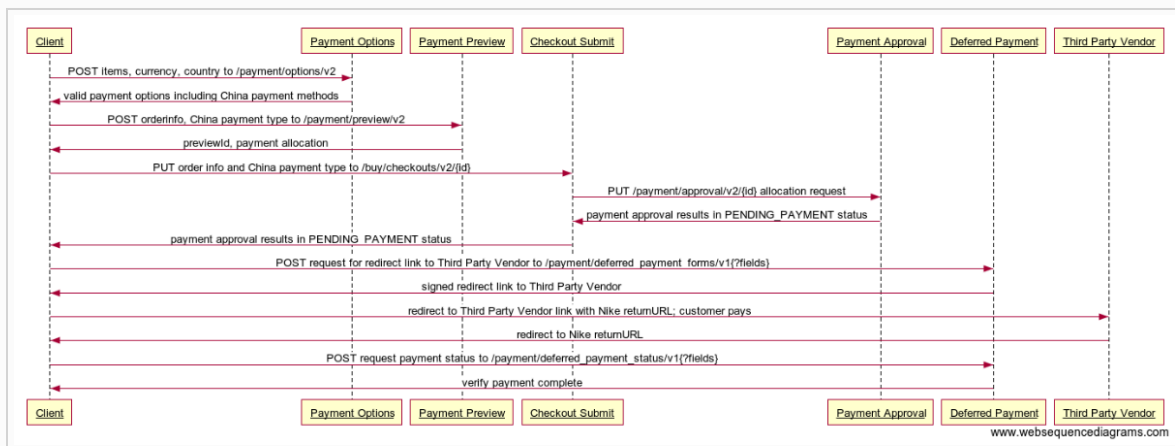
Sample Apple Pay flow

In the example Payment API flow below, the customer chooses to pay by Apple Pay in a Safari web browser.



Sample Deferred Payment flow

In this flow, the customer chooses to pay by a payment method that will be authorized and captured after the Nike order has been placed. This is a typical flow for China payment methods such as WeChat and Alipay.



Polling

To avoid excessive job polling of asynchronous endpoints, wait the number of milliseconds returned in the job request eta before checking the job status.

Retry Conditions

For all Payment APIs, the general rule is that requests resulting in a HTTP 4XX response should not be retried without modification to the request data, but HTTP 5XX errors can be retried as is. For general information on Nike error retry practices, see [API Error Patterns](#) on Confluence.

The exception to the 4xx response rule is the 429 response, indicating that there are too many requests coming in for the service to handle. When a service returns a 429, the call should be retried a maximum of two times, using the value returned in the `Retry-After`

header to determine when to make the follow-up call.

Troubleshooting

Listed below are some techniques to troubleshoot problems using the Payment API.

Query Splunk With a Trace ID

In order to abide by PCI-compliance rules, payment logging requires special Splunk access. As a result, you can not query Splunk by Trace ID as a trouble-shooting tool to track down why a payment request failed. If need help from the Payment Team, post your problem to the [#cic-payment](#) Slack channel with details such as:

- experience in which you encountered the error, iOS SNKRS app, nike.com web, Android SNKRS app, direct endpoint call etc.
- time request failed
- Trace ID
- request URI and body (if not GET request)
- error codes and error messages

Inspect Browser Activity in a Live Experience

Try using your browser's built-in tools for inspecting web service calls made from a live Nike experience such as [SNKRS Web](#). Or, set up Charles and your favorite device to proxy service calls made from the Nike SNKRS or Nike+ Apps. Sometimes seeing what other experiences are doing might address your question or concern.

TIP: While inspecting <http://www.nike.com/launch>, you can change your shopping country with the flag icon at the upper right of the homepage to

test different locales. Place orders in different countries with different payment methods to view the Payment call flow with other CiC services. Orders can be cancelled via self-service within 30 minutes of submission, otherwise contact Nike customer Service.

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of the [Using NDe APIs](#) guide on how to create and register your caller ID.

Authentication

Access Tokens

Most calls through the Nike API gateway (api.nike.com) require an access token be sent in the request header. This allows Nike to verify that your experience is authorized to perform the action on behalf of the user.

Access tokens are obtained by calling Nike Unite services prior to calling the API which you ultimately want to reach.

To find out more on how to call Unite services to obtain access tokens, see the Authorization section of the [Using NDe APIs](#) guide.

JSON Web Token

A few of the endpoints in the Payment APIs require the additional authorization of a JSON Web Token (JWT). For more, see the JWT section of [Using NDe APIs](#).

Contacting the Team

Need to contact the Payment team?

Slack	#cic-payment
Confluence Space	CiC Payment
Product Owner	Sree Krishna

Glossary

Term	Definition
Authorization	Temporary hold on funds in a customer's account for a future charge
Credit	Funds that are added to a customer's account
Debit	Funds that are removed from a customer's account
Deferred Payment	Payment made after the Nike order after is placed
DOMS	Distributed Order Management System, also known as Sterling that handles order fulfillment
ESB	Enterprise Service Bus, similar to PAC but used to communicate with Nike non-commerce systems
PAC	Messaging system used by DOMS to communicate with other Nike commerce systems

Term	Definition
PCI-DSS	Payment Card Industry Data Security Standard provides secure standards for handling credit card data. All Nike CiC payment services are PCI-DSS compliant.
Reauthorization	Temporary hold of funds in a customer's account is reissued, typically when the original authorization has expired
Void	Reverses a successful Authorization, also known as an authorization reversal

Supported Stored Payment Types

The Stored Payment Service supports storing these types of payment:

Payment Type Description	Value	Storage Limit
Alipay	AliPay	1
Apple Pay	ApplePay	1
Credit Card	CreditCard	4
Gift Card	GiftCard	10
PayPal	PayPal	1
Tenpay	TenPay	1
UnionPay	UnionPay	1
WeChat	WeChat	1

Payment Options by Country

See the [Global Payment Options](#) for a list of supported payment types by shipping and billing country.

Document Change Log

Summary	Date	Description
Initial draft	01/08/2018	Initial Draft
Payment Gateway	11/26/2018	Added Payment Gateway Topic
Use Cases	2/21/2018	Restructured for use cases

Next Steps

You've now learned how to add payment to your experience. Here are some next steps.

- [Capturing User Events](#)
- [Adding Order History to your experience](#)
- [Using NDe APIs](#)